

CS4423 - Networks

Prof. Götz Pfeiffer

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

1. Graphs and Graph Theory

Lecture 3: Graphs, Relations and a Matrix

Start by importing the `networkx` package. Also import `numpy` for matrix operations.

```
In [1]: import networkx as nx  
import numpy as np
```

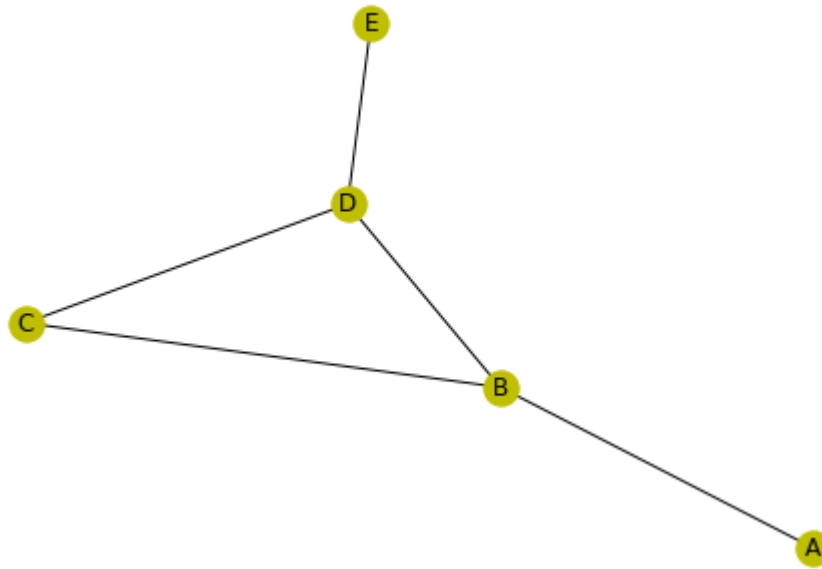
We've seen how the mathematical concept of a (simple) graph can serve as a (mathematical) model of a network.

Here, we're going to relate graphs to two other [areas of mathematics](https://en.wikipedia.org/wiki/Areas_of_mathematics) (https://en.wikipedia.org/wiki/Areas_of_mathematics):

- **Linear Algebra** and its language of matrices and **matrix multiplication**
- **Set Theory** and the language of **relations**

As before, let's construct a small graph G .

```
In [2]: G = nx.Graph(["AB", "BC", "BD", "CD", "DE"])
opts = { "with_labels": True, "node_color": 'y' }
nx.draw(G, **opts)
```



Adjacency Matrix

A useful algebraic way to represent a graph is given by its **adjacency matrix**. This is square matrix A , with rows and columns corresponding to the vertices of the graph, and an entry 1 or 0 in row i , column j , if the corresponding vertices are joined by an edge or not. The edge AB in the example above this gives an entry 1 in row 1 (corresponding to vertex A) and column 2 (corresponding to vertex B). And another entry 1 in row 2 column 1. The full adjacency matrix of the above graph is as follows.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

In `networkx`, the adjacency matrix is computed with the `adjacency_matrix` command.

```
In [3]: A = nx.adjacency_matrix(G)
print(A)
```

```
(0, 1)      1
(1, 0)      1
(1, 2)      1
(1, 3)      1
(2, 1)      1
(2, 3)      1
(3, 1)      1
(3, 2)      1
(3, 4)      1
(4, 3)      1
```

This matrix is internally represented as a `scipy` sparse matrix (as in general many of its entries are 0) and needs to be converted (e.g. by the `toarray` command) in order to be displayed as a proper matrix.

(Note: the `todense` command converts an adjacency matrix to a `numpy` `matrix`. But this class is being deprecated ...

So we use `toarray` to convert an adjacency matrix to a `numpy` `ndarray`. But here, `*` doesn't mean matrix multiplication ...)

```
In [4]: type(A)
```

```
Out[4]: scipy.sparse.csr.csr_matrix
```

```
In [5]: type(A.toarray())
```

```
Out[5]: numpy.ndarray
```

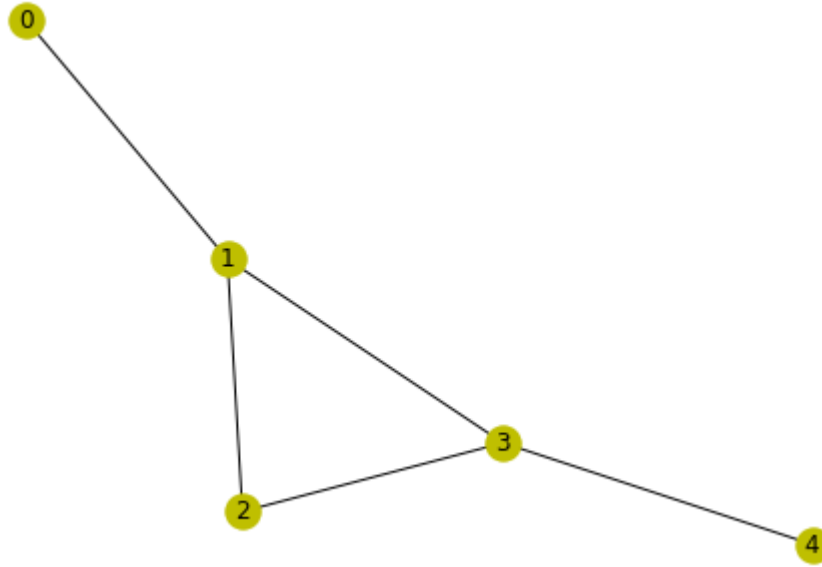
```
In [6]: print(A.toarray())
```

```
[[0 1 0 0 0]
 [1 0 1 1 0]
 [0 1 0 1 0]
 [0 1 1 0 1]
 [0 0 0 1 0]]
```

Note that the matrix $A = (a_{ij})$, like every adjacency matrix of a simple graph, is **symmetric** (about the diagonal): $a_{ij} = a_{ji}$ for all i, j . Also, all diagonal entries are 0.

Going the other way, a graph can be constructed from an adjacency matrix (at the cost of losing the node labels ...)

```
In [7]: H = nx.from_numpy_matrix(A.toarray())
        nx.draw(H, **opts)
```



Some examples of matrix multiplication with `scipy` sparse matrices ... we'll get back to that later.

```
In [8]: print((A**0).toarray())
```

```
[[1 0 0 0 0]
 [0 1 0 0 0]
 [0 0 1 0 0]
 [0 0 0 1 0]
 [0 0 0 0 1]]
```

```
In [9]: print((A*A).toarray())
```

```
[[1 0 1 1 0]
 [0 3 1 1 1]
 [1 1 2 1 1]
 [1 1 1 3 0]
 [0 1 1 0 1]]
```

Degree

The **degree** of a vertex x in a simple graph is the number of vertices it is connected to in the graph (it's number of **neighbours**, or **friends**). The degree can serve as a (simple) measure of the importance of a node in a network.

Fact. The **average degree** of the nodes in a network depends (only) on the number n of nodes, and the number m of edges in the network.

This can be seen as follows: Writing k_i for the degree of vertex x_i , this number can easily be determined from the adjacency matrix A as the number of entries 1 in row i (or in column i):

$$k_i = \sum_j a_{ij} = \sum_j a_{ji}.$$

As every edge contributes to the degree of 2 nodes, the sum of all degrees equals twice the number of edges:

$$\sum_i k_i = \sum_{i,j} a_{ij} = 2m,$$

whence the **average degree** is

$$c = \frac{1}{n} \sum_i k_i = \frac{2m}{n}.$$

As a consequence, any simple graph G has an even number of nodes of odd degree. This fact is known as [Euler's Handshake Lemma](https://en.wikipedia.org/wiki/Handshaking_lemma) (https://en.wikipedia.org/wiki/Handshaking_lemma).

In our graph G , the column sums of the adjacency matrix A are:

```
In [10]: A.toarray().sum(0)
```

```
Out[10]: array([1, 3, 2, 3, 1])
```

and the row sums are:

```
In [11]: A.toarray().sum(1)
```

```
Out[11]: array([1, 3, 2, 3, 1])
```

and both agree with the degrees of the nodes of G :

```
In [12]: G.degree()
```

```
Out[12]: DegreeView({'A': 1, 'B': 3, 'C': 2, 'D': 3, 'E': 1})
```

The sum of the degrees is 10, the average degree is $\frac{2m}{n} = 2$, and there are 4 nodes of odd degree.

```
In [13]: A.sum()
```

```
Out[13]: 10
```

Graphs are Relations

Recall the following definitions.

Definition. A **relation** from a set X to a set Y is (nothing but) a subset R of the Cartesian product $X \times Y = \{(x, y) : x \in X, y \in Y\}$. We say that $x \in X$ is **R -related** to $y \in Y$ whenever $(x, y) \in R$ and then write xRy .

- The **adjacency matrix** of a relation $R \subseteq X \times Y$ is the matrix with one row for each element $x \in X$ and one column for each element $y \in Y$, and it has an entry **1** in row x and column y if xRy , and entries **0** otherwise.
- In many cases, $Y = X$, i.e., R is a **homogeneous** relation. In this case, we say that R is a relation **on** X . Such a relation can have one or more of the following properties.

- (R) R is **reflexive** if xRx for all $x \in X$;
- (S) R is **symmetric** if xRy implies yRx for all $x, y \in X$;
- (T) R is **transitive** if xRy and yRz imply that xRz for all $x, y, z \in X$;
- (I) R is **irreflexive** if not xRx for all $x \in X$;
- (A) R is **antisymmetric** if xRy and yRx imply that $x = y$ for all $x, y \in X$.

- A relation that is (RST), i.e., reflexive, symmetric and transitive, is called an **equivalence relation**, and it partitions the set X into (mutually disjoint) parts, called **equivalence classes**.
- A relation that is (RAT) is called a **partial order** (such as the **divides** partial order on the natural numbers, or the **contains** relation between the subsets of a set).
- In view of these notions, we can now describe simple graphs and some of their properties as follows: A *simple graph* with node set X is a *symmetric, irreflexive* relation on X .
- (A *directed graph* with node set X is *irreflexive* if it has *no loops*. And it is *antisymmetric* if every edge has a *unique direction*.)

The article [Counting Transitive Relations](https://cs.uwaterloo.ca/journals/JIS/VOL7/Pfeiffer/pfeiffer6.html)

(<https://cs.uwaterloo.ca/journals/JIS/VOL7/Pfeiffer/pfeiffer6.html>) (in the *Journal of Integer Sequences*) contains a systematic account on the various types of relations that can be distinguished by these 5 properties, and a discussion of how to count them (up to equivalence) in case the underlying set X is finite.

Composition and Adjacency Matrices.

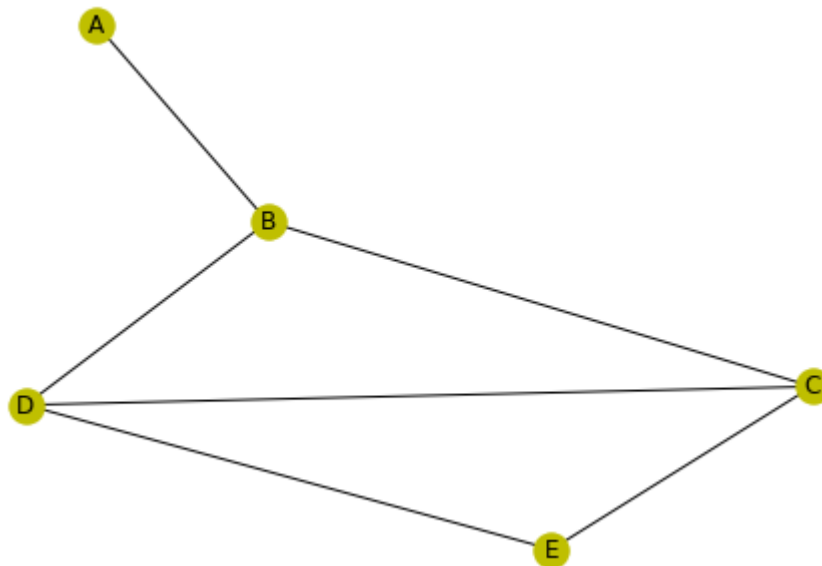
- Relations can be composed, like functions.

- If R is a relation from a set X to a set Y , and if S is a relation from Y to a set Z , then the **composite relation** $R \circ S$ is the relation from X to Z , defined by $x(R \circ S)z$ if there is an element $y \in Y$ such that xRy and ySz .
- The adjacency matrix of the composite relation $R \circ S$ is essentially the (matrix) product of the adjacency matrices of the individual relations R and S .
- If $A = (a_{ij})$ is the adjacency matrix of R , and $B = (b_{jk})$ the adjacency matrix of S , then the i, k -entry of the product AB is

$$(AB)_{ik} = \sum_j a_{ij}b_{jk},$$
 which is exactly the **number** of elements $y \in Y$ such that $x_i R y$ and $y S z_k$.
- All it needs for x_i to be $(R \circ S)$ -related to z_k is this number to be at least 1.
- Hence, replacing all nonzero entries in the product matrix AB with 1 yields the adjacency matrix of the composite $R \circ S$.
- Recall that a graph G on a vertex set X is the same as a symmetric irreflexive relation R on the set X .
- We can form the matrix product of the adjacency matrix A of G (or R) with itself. What is the **meaning** of the entries of that product?

For example:

```
In [14]: G = nx.Graph(["AB", "BC", "BD", "CD", "CE", "DE"])
          nx.draw(G, **opts)
```



```
In [15]: A = nx.adjacency_matrix(G).toarray()
AA = A @ A
print(AA)

[[1 0 1 1 0]
 [0 3 1 1 2]
 [1 1 3 2 1]
 [1 1 2 3 1]
 [0 2 1 1 2]]
```

In numpy , one can use **boolean indexing** and other convenient methods to convert A^2 into an adjacency matrix of a graph.

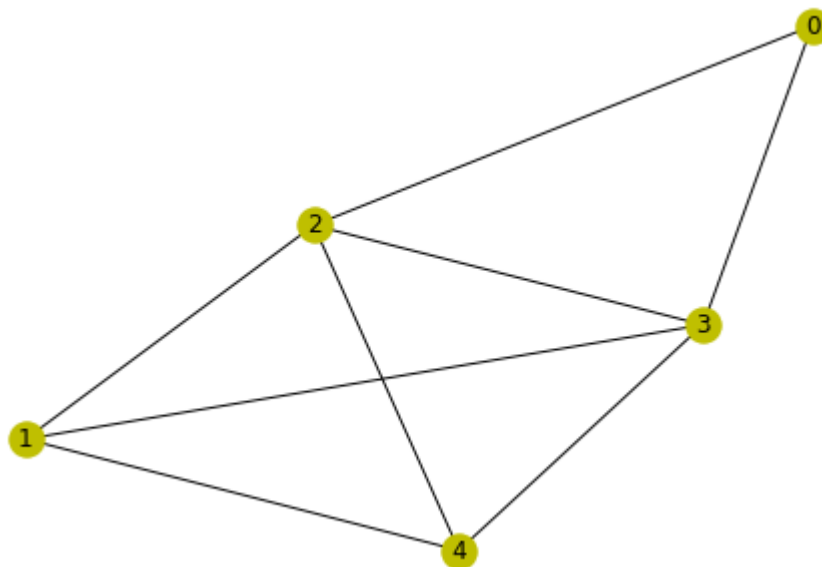
```
In [16]: AA[AA>1] = 1
print(AA)

[[1 0 1 1 0]
 [0 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 1 1 1 1]]
```

```
In [17]: np.fill_diagonal(AA, 0)
print(AA)

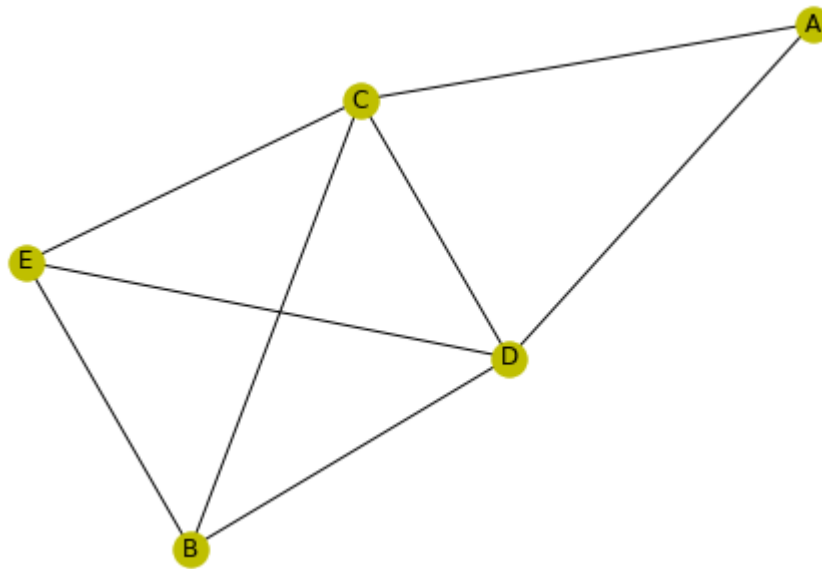
[[0 0 1 1 0]
 [0 0 1 1 1]
 [1 1 0 1 1]
 [1 1 1 0 1]
 [0 1 1 1 0]]
```

```
In [18]: GG = nx.from_numpy_matrix(AA)
nx.draw(GG, **opts)
```



- Oops - The node names got lost. They can be revived by relabelling the nodes in GG .

```
In [19]: nx.relabel_nodes(GG, { i : "ABCDE"[i] for i in range(5)}, copy=False)
nx.draw(GG, **opts)
```



Code Corner

Numpy

- `sum` : form the sum of matrix entries, either all or along a specified axis
- `toarray` : convert a sparse matrix into a proper array
- `fill_diagonal` : fill the diagonal entries of an array with a given value.

networkx

- `adjacency_matrix` computes the adjacency matrix of a graph
- `from_numpy_matrix` constructs a graph from its adjacency matrix

Exercises

1. Use the `complete_graph` function in `networkx` to construct a 5×5 matrix with entries 0 on the diagonal and all other entries 1.

In []: